

```
import json, unittest, datetime

# Sample data for testing
jsonData1 = {
    "deviceID": "dh28dslkja",
    "deviceType": "LaserCutter",
    "timestamp": 1624445837783,
    "location": "japan/tokyo/keiyō-
industrial-zone/daikibo-factory-
meiyo/section-1",
    "operationStatus": "healthy",
    "temp": 22
}

jsonData2 = {
    "device": {
        "id": "dh28dslkja",
        "type": "LaserCutter"
    },
    "timestamp": "2021-06-
23T10:57:17.783Z",
    "country": "japan",
    "city": "tokyo",
    "area": "keiyō-industrial-zone",
    "factory": "daikibo-factory-meiyo",
    "section": "section-1",
    "data": {
        "status": "healthy",
        "temperature": 22
    }
}
```

```

}

jsonExpectedResult = {
  "deviceID": "dh28dslkja",
  "deviceType": "LaserCutter",
  "timestamp": 1624445837783,
  "location": {
    "country": "japan",
    "city": "tokyo",
    "area": "keiyō-industrial-zone",
    "factory": "daikibo-factory-meio",
    "section": "section-1"
  },
  "data": {
    "status": "healthy",
    "temperature": 22
  }
}

# Convert Format 1 to expected format
def convertFromFormat1(jsonObject):
  locationParts =
    jsonObject['location'].split('/')
  result = {
    'deviceID': jsonObject['deviceID'],
    'deviceType':
      jsonObject['deviceType'],
    'timestamp':
      jsonObject['timestamp'],
    'location': {
      'country': locationParts[0],
      'city': locationParts[1],
      'area': locationParts[2],

```

```

        'factory': locationParts[3],
        'section': locationParts[4]
    },
    'data': {
        'status':
jsonObject['operationStatus'],
        'temperature':
jsonObject['temp']
    }
}
return result

# Convert Format 2 to expected format
def convertFromFormat2(jsonObject):
    data =
datetime.datetime.strptime(jsonObject['time
stamp'], '%Y-%m-%dT%H:%M:%S.%fZ')
    timestamp = round((data -
datetime.datetime(1970, 1,
1)).total_seconds() * 1000)

    result = {
        'deviceID': jsonObject['device']
['id'],
        'deviceType': jsonObject['device']
['type'],
        'timestamp': timestamp,
        'location': {
            'country':
jsonObject['country'],
            'city': jsonObject['city'],
            'area': jsonObject['area'],
            'factory':

```

```

        jsonObject['factory'],
            'section':
jsonObject['section']
        },
        'data': jsonObject['data']
    }
    return result

def main(jsonObject):
    result = {}
    if jsonObject.get('device') is None:
        result =
convertFromFormat1(jsonObject)
    else:
        result =
convertFromFormat2(jsonObject)
    return result

# Unit Tests
class TestSolution(unittest.TestCase):

    def test_sanity(self):
        result =
json.loads(json.dumps(jsonExpectedResult))
        self.assertEqual(result,
jsonExpectedResult)

    def test_dataType1(self):
        result = main(jsonData1)
        self.assertEqual(result,
jsonExpectedResult, 'Converting from Type 1
failed')

```

```
def test_dataType2(self):  
    result = main(jsonData2)  
    self.assertEqual(result,  
jsonExpectedResult, 'Converting from Type 2  
failed')  
  
if __name__ == '__main__':  
    unittest.main()
```